



机器学习-实验作业 1

李汶峰

23 级网安 1 班

202300460158

2025 年 10 月 23 日

目录

1 概述	2
1.1 实验目的	2
1.2 实验环境	2
1.3 实验方法	2
2 数据预处理	2
2.1 数据集概述	2
2.2 数据加载及初步清洗	3
2.3 数据重组织	3
2.4 Z-score 数据标准化	4
2.5 数据集划分及偏置项	4
3 特征选取	5
3.1 仅使用 PM2.5	5
3.2 使用全特征	6
3.3 使用高相关性特征	6
4 梯度下降技术选取	8
4.1 BGD	8
4.2 Adagrad	9
4.3 Adam	10
5 调整学习率	11
6 正则化引入	13
7 结果分析与评估	14
8 结论	16
9 附录	16

1 概述

1.1 实验目的

在本实验中，基于训练数据 (train.csv) 使用线性回归模型预测测试数据 (test.csv) 中的 PM2.5 值。train.csv 包含每个月前 20 天的完整观测数据；test.csv 是从剩余数据中采样出的 240 笔不重复的数据，每笔数据使用前 9 小时的观测作为特征 (feature)，第 10 小时的 PM2.5 作为答案 (answer)。其中数据里共有 18 项不同的特征。

1.2 实验环境

- 硬件：AMD Ryzen 7 7735H 处理器，16GB 内存，Windows11 系统
- 软件：python3.13.7
- 编译器：pycharm
- 用到的库：pandas、numpy、math、time
- 训练集 train.csv、测试集 test.csv

1.3 实验方法

本研究采用“由简入深”的分阶段策略，逐步优化用于 PM2.5 预测的线性回归模型。首先构建一个基线模型：仅以 PM2.5 自身的近 9 小时历史作为输入，使用批量梯度下降 (SGD) 训练，建立最简对照模型。其后引入全部 18 项观测指标作为输入，评估所有特征带来的性能变化。在此基础上，使用 Pearson 相关系数计算出高相关性指标作为特征输入到模型之中，比较计算效率与泛化表现。随后，替换优化策略，分别采用 Adagrad 与 Adam 等自适应优化方法，系统评估不同梯度下降技术对收敛速度与稳定性的影响。接着，针对学习率进行对比实验，观察四种不同学习率对收敛曲线与验证集表现，从而选取最优学习率。最后，在损失函数中加入 L2 正则化，以抑制权重过大并降低过拟合风险。各阶段均采用 80% 训练集和 20% 验证集划分，统一以验证集 RMSE 为评估指标，训练迭代 10,000 次，并控制整体运行时间在 3 分钟以内。

2 数据预处理

数据预处理是机器学习流程中的关键环节，直接影响模型的训练效果和预测性能。本实验针对 PM2.5 预测任务，设计并实施了一套完整的数据预处理流程，包括数据加载、缺失值处理、特征构建、数据标准化以及数据集划分等步骤。

2.1 数据集概述

训练集 (train.csv)

- 时间范围：每个月的前 20 天，每天 24 小时
- 特征维度：18 种空气质量观测指标（包括 PM2.5、PM10、SO2、CO、O3、NO2 等）
- 数据规模：12 月 $\times$ 20 天 $\times$ 24 小时 $\times$ 18 特征 = 103,680 个数据点

测试集 (test.csv)

- 数据格式: 240 个测试样本, 每个样本包含连续 10 小时的观测数据
- 任务目标: 根据前 9 小时的所有观测数据预测第 10 小时的 PM2.5 浓度

2.2 数据加载及初步清洗

使用 pandas 库读取 CSV 文件, 采用 utf-8-sig 编码方式以正确处理中文字符和 BOM 标记:

```
1 data = pd.read_csv('set/train.csv', encoding='utf-8-sig')
```

原始数据的前 3 列 (日期、站点名称、测项名称) 为元数据, 对模型训练无直接贡献, 因此将其移除, 仅保留 24 小时的数值型观测数据。使用以下方式进行处理, 处理后每行包含 24 列数值数据, 对应 24 小时的观测值。

```
1 data = data.iloc[:, 3:].copy()
```

降雨量特征中的 NR 表示“无降雨”, 为了适合机器学习, 将 NR 全部改成 0。

```
1 data[data == 'NR'] = 0
```

2.3 数据重组织

将原始数据按月份组织成标准化的三维数据结构, 每月数据组织为 18×480 的矩阵, 其中行维度对应 18 种空气质量观测指标、列维度对应 20 天 $\times$ 24 小时的时间序列, 代码如下:

```
1 month_data = {}
2 for month in range(12):
3     sample = np.empty([18, 480])
4     for day in range(20):
5         sample[:, day * 24: (day + 1) * 24] =
6             raw_data[18*(20*month + day): 18*(20 * month + day + 1), :]
7     month_data[month] = sample
```

采用滑动窗口方法构建训练样本, 窗口大小为 9 小时的特征和 1 小时的目标。从 12 个月的时间序列数据中构建训练集: 以每小时为步长滑动一个 10 小时的时间窗口, 提取前 9 小时的特征作为输入 x , 提取第 10 小时的 PM2.5 浓度作为预测目标 y , 每月生成 471 个样本, 最终生成 5652 个训练样本。

```
1 selected_features = [9]
2 x = np.empty([12 * 471, len(selected_features) * 9], dtype=float)
3 y = np.empty([12 * 471, 1], dtype=float)
4 for month in range(12):
5     for day in range(20):
6         for hour in range(24):
7             if day == 19 and hour > 14:
8                 continue
9             x[month * 471 + day * 24 + hour, :] =
```

```
10     month_data[month][selected_features, day * 24
11     + hour: day * 24 + hour + 9].reshape(1, -1)
12     y[month * 471 + day * 24 + hour, 0] =
13     month_data[month][9, day * 24 + hour + 9]
```

2.4 Z-score 数据标准化

特征矩阵 x 包含了每个样本的前 9 小时的相关特征观测值，这些值可能存在量纲不一致或尺度差异的问题。为解决这一问题并提升梯度下降算法的训练效率和数值稳定性，我们对特征矩阵 x 应用 Z-score 标准化。这一预处理步骤的核心目的是将每个特征缩放到均值为 0、标准差为 1 的分布，从而使所有特征在相同尺度上贡献梯度，避免某些特征主导优化过程。计算每个特征的均值 μ 和标准差 σ ，然后应用变换 $x' = (x - \mu)/\sigma$ ：

```
1 mean_x = np.mean(x, axis=0)
2 std_x = np.std(x, axis=0)
3 for i in range(len(x)):
4     for j in range(len(x[0])):
5         if std_x[j] != 0:
6             x[i][j] = (x[i][j] - mean_x[j]) / std_x[j]
```

2.5 数据集划分及偏置项

训练样本按 80%-20% 比例顺序划分训练集（约 4522 样本）和验证集（约 1130 样本）。线性回归模型的标准形式为 $y = Xw + b$ ，其中 b 为偏置项。在矩阵表示中， b 可融入权重向量 w ，通过在特征矩阵 X 的首列添加全 1 向量实现： $X' = [1, X]$ ， $w' = [b, w]$ 。这种技巧将模型简化为 $y = X'w'$ ，无需单独参数化 b ，并便于 NumPy 的矩阵乘法优化。在本实现中，在数据集划分后立即为训练集和验证集添加偏置列：

```
1 split_idx = math.floor(len(x) * 0.8)
2 x_train = x[:split_idx, :]
3 y_train = y[:split_idx, :]
4 x_validation = x[split_idx:, :]
5 y_validation = y[split_idx:, :]
6
7 x_train = np.concatenate(
8     (np.ones([len(x_train), 1]), x_train), axis=1).astype(float)
9 x_validation = np.concatenate(
10    (np.ones([len(x_validation), 1]), x_validation), axis=1).astype(float)
```

3 特征选取

在本任务中，特征选取是提升模型性能的关键步骤。原始数据包含 18 项观测指标，每笔数据使用前 9 小时的观测作为输入预测第 10 小时的 PM2.5 值。在特征选取里，若高维特征可能引入噪声，导致过拟合或计算开销增加；特征较少则可能导致欠拟合。本实验通过三种特征选取方法对比模型性能：仅使用 PM2.5 作为单一特征、使用所有 18 个特征、基于 Pearson 相关系数选取高相关性特征。目的在于量化特征数量对线性回归模型的 RMSE、收敛速度和运行时间的影响。

为了保证测量不同特征选取对结果的影响，采用控制变量法，仅对特征进行修改而其他不变。

3.1 仅使用 PM2.5

首先仅使用 PM2.5 作为特征值，忽略其他指标。作为自回归基线，假设 PM2.5 历史值能预测未来值。由于 PM2.5 的索引位于 9 号位置，所以使用 `selected_features = [9]` 来作为特征选择。

```
1 selected_features = [9]
2 x = np.empty([12 * 471, len(selected_features) * 9], dtype=float)
3 # 提取前 9 小时 PM2.5
```

运行结果如下图所示，在仅使用 PM2.5 作为特征的情况下，经过 10000 次循环后，RMSE loss 已经降低到 6.1940，最终的验证集上 RMSE 为 5.8619，总运行时间 1.74 秒。但由于仅依赖 PM2.5 的历史值（9 维特征），信息有限，导致最终损失较高而欠拟合。

```
Iteration 0: Train RMSE = 27.2396, Val RMSE = 26.3874
Iteration 1000: Train RMSE = 6.3787, Val RMSE = 6.1848
Iteration 2000: Train RMSE = 6.2457, Val RMSE = 5.9683
Iteration 3000: Train RMSE = 6.2101, Val RMSE = 5.9065
Iteration 4000: Train RMSE = 6.1992, Val RMSE = 5.8830
Iteration 5000: Train RMSE = 6.1957, Val RMSE = 5.8727
Iteration 6000: Train RMSE = 6.1946, Val RMSE = 5.8676
Iteration 7000: Train RMSE = 6.1942, Val RMSE = 5.8648
Iteration 8000: Train RMSE = 6.1941, Val RMSE = 5.8633
Iteration 9000: Train RMSE = 6.1941, Val RMSE = 5.8624
```

最终模型评估：

验证集 RMSE: 5.8619

总运行时间: 1.74 秒

迭代次数: 10000

最终训练RMSE: 6.1940

最终验证RMSE: 5.8619

图 1: 仅使用 PM2.5 的训练结果图

3.2 使用全特征

接着使用全部的指标作为特征，捕捉污染物、气象等多方面影响。假设完整信息提升预测精度，但可能引入一些低相关的噪声。

```
1 selected_features = list(range(18))
2 x = np.empty([12 * 471, len(selected_features) * 9], dtype=float)
3 # 提取所有特征
```

运行结果如下图所示，在使用所有指标作为特征的情况下，最终训练集上 RMSE 为 5.7213、验证集上 RMSE 为 5.6652，总运行时间 5.56 秒。全特征模型相比仅使用 PM2.5 显著提升了精度，但高维特征空间可能会带来计算时间大幅增长、模型过拟合等问题。

训练集样本数：4521

验证集样本数：1131

```
Iteration 0: Train RMSE = 27.2396, Val RMSE = 26.3874
Iteration 1000: Train RMSE = 5.9242, Val RMSE = 5.9413
Iteration 2000: Train RMSE = 5.7959, Val RMSE = 5.7402
Iteration 3000: Train RMSE = 5.7537, Val RMSE = 5.6870
Iteration 4000: Train RMSE = 5.7365, Val RMSE = 5.6703
Iteration 5000: Train RMSE = 5.7290, Val RMSE = 5.6652
Iteration 6000: Train RMSE = 5.7254, Val RMSE = 5.6641
Iteration 7000: Train RMSE = 5.7235, Val RMSE = 5.6642
Iteration 8000: Train RMSE = 5.7224, Val RMSE = 5.6646
Iteration 9000: Train RMSE = 5.7218, Val RMSE = 5.6649
```

最终模型评估：

验证集 RMSE：5.6652

总运行时间：5.44 秒

最终训练RMSE：5.7213

最终验证RMSE：5.6652

图 2: 使用全特征的训练结果图

3.3 使用高相关性特征

为了减少特征维度同时减少计算时间、防止过拟合问题，需要选取与 PM2.5 相关性较大的指标作为特征，而不必选取全部指标。本实验中，首先使用 Pearson 相关系数计算各个指标和 PM2.5 之间的相关性，选取相关性较大的作为特征。

Pearson 相关系数用于衡量两个连续变量之间的线性相关程度，其计算公式如下

$$r_{X,Y} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} = \frac{\text{cov}(X,Y)}{\sigma_X \sigma_Y},$$

其中： x_i, y_i ：第 i 个观测样本的 X 和 Y 值； $\bar{x}, \bar{y}$ ： X 和 Y 的均值； $\text{cov}(X,Y)$ ：协方差； σ_X, σ_Y ：标准差； n ：样本数量。

r 的取值范围为 $[-1, 1]$: $r = 1$ 表示完全正相关, $r = -1$ 完全负相关, $r = 0$ 无线性相关。我们使用 SciPy 库的 `pearsonr` 函数计算 r 和 p-value (显著性检验), 阈值设为 $|r| > 0.3$ 且 p-value < 0.05 , 确保统计显著性。

在实验中, 我们从 `train.csv` 提取各特征列 (每 18 行一组, 跨月展平), 计算与 PM2.5 (索引 9) 的相关系数。代码实现如下:

```
1 # 计算 Pearson 相关系数
2 pm25_idx = 9
3 pm25_values = raw_data[pm25_idx::18].flatten()
4 correlations = []
5
6 for i in range(18):
7     feature_values = raw_data[i::18].flatten()
8     corr, p_value = pearsonr(feature_values, pm25_values)
9     correlations.append((i, feature_names[i], corr, p_value))
```

获得的结果如下图所示:

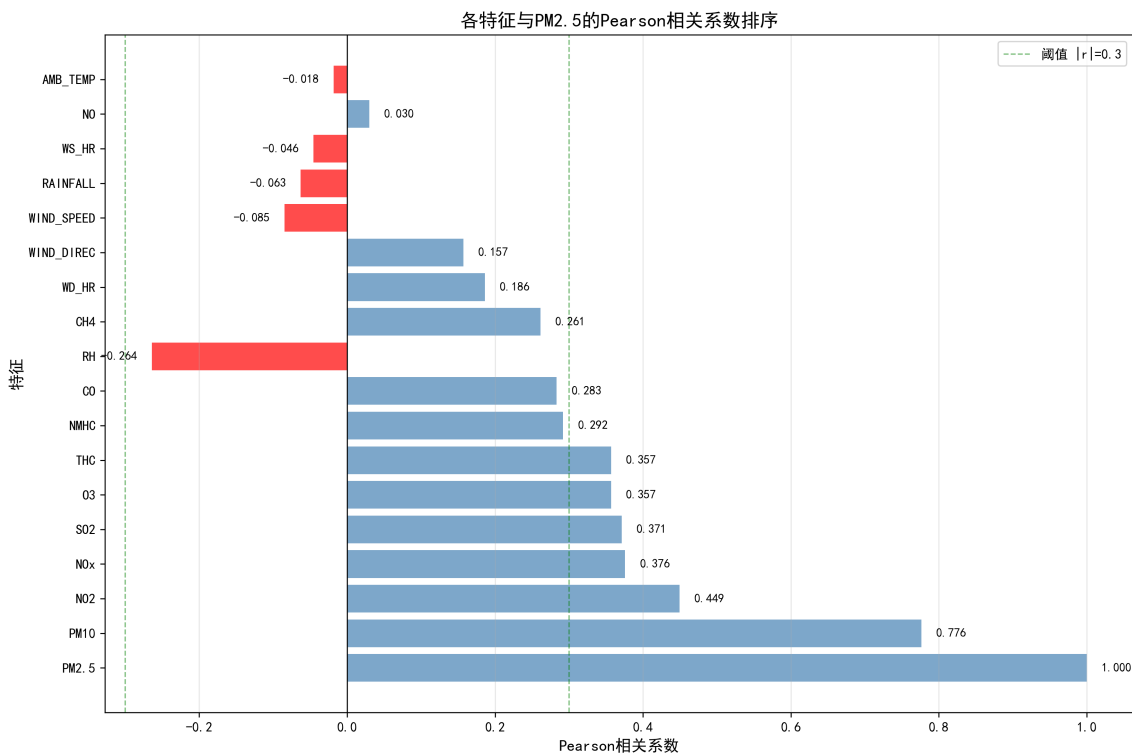


图 3: 相关性计算排序

从上图可以看出, PM2.5、PM10、NO_x、NO₂、SO₂、O₃ 和 THC 这七个指标与目标变量的相关系数绝对值最高, 而其他指标的相关性较弱或呈负相关, 因此我们选取这七个高相关指标作为核心特征, 以最大化信息增益并最小化噪声干扰。下图展示了使用这些特征的模型运行结果: 最终训练集 RMSE 为 5.7950、验证集 RMSE 为 5.6267, 总运行时间仅 2.90 秒。与仅使用 PM2.5 的模型相比, 该方法显著提升了预测精度; 与全特征模型相比, RMSE 变化不大, 但运行时间大幅缩短, 体现了高效的特征筛选优势。基于此, 我们最终选用 PM2.5、PM10、NO_x、NO₂、SO₂、O₃

和 THC 作为后续实验的标准特征集，以平衡模型准确性和计算效率。

```
Iteration 0: Train RMSE = 27.2396, Val RMSE = 26.3874
Iteration 1000: Train RMSE = 5.9880, Val RMSE = 5.9029
Iteration 2000: Train RMSE = 5.8620, Val RMSE = 5.7035
Iteration 3000: Train RMSE = 5.8219, Val RMSE = 5.6504
Iteration 4000: Train RMSE = 5.8063, Val RMSE = 5.6333
Iteration 5000: Train RMSE = 5.7998, Val RMSE = 5.6278
Iteration 6000: Train RMSE = 5.7971, Val RMSE = 5.6262
Iteration 7000: Train RMSE = 5.7959, Val RMSE = 5.6260
Iteration 8000: Train RMSE = 5.7954, Val RMSE = 5.6262
Iteration 9000: Train RMSE = 5.7951, Val RMSE = 5.6265
```

最终模型评估:

验证集 RMSE: 5.6267

总运行时间: 2.90 秒

最终训练RMSE: 5.7950

最终验证RMSE: 5.6267

图 4: 使用高相关性特征训练结果

4 梯度下降技术选取

在本次线性回归模型中，优化器的选取是提升训练效率和收敛稳定性的关键。传统随机梯度下降（SGD）学习率固定，易受尺度不均影响导致收敛慢或震荡。为解决此问题，我们比较并选取自适应优化器 Adagrad 和 Adam 作为备选方案。这些方法通过动态调整每个参数的学习率，适应高维空气质量数据（如 PM2.5 预测任务）的非平稳性和噪声特性。实验中，我们在阶段 4 引入这些优化器，固定学习率 $=0.1$ 、迭代 10000 次，评估验证集 RMSE 和损失曲线。最终，Adagrad 在平衡收敛速度与计算开销方面表现最佳，用于后续实验。

4.1 BGD

线性回归模型的参数 w 通过梯度下降法优化，以最小化均方误差（MSE）损失函数。采用全批量梯度下降，即每次迭代使用整个训练集计算精确梯度。这种方法确保梯度无偏且收敛平稳，适合本实验的中小规模数据集。其是经典梯度下降的直接形式，适用于凸优化问题如线性回归，其中损失函数具有唯一全局最小值。

损失函数定义为：

$$L(w) = \frac{1}{N} \sum_{i=1}^N (y_i - x_i^T w)^2 = \frac{1}{N} \|y - Xw\|_2^2$$

其梯度为：

$$\nabla L(w) = \frac{2}{N} X^T (Xw - y)$$

参数更新规则为:

$$w \leftarrow w - \eta \nabla L(w)$$

其中 η 为固定学习率, N 为训练样本数, X 为带偏置的特征矩阵。最大迭代次数 $T = 10000$, 足以实现收敛。

```
1 for t in range(iter_time):
2     y_pred = np.dot(x_train, w)
3     gradient = 2 * np.dot(x_train.T, (y_pred - y_train)) / len(x_train)
4     # 更新规则
5     w -= learning_rate * gradient
```

从图 4 中可以看出, 在使用 BGD 时, 最终训练集 RMSE 为 5.7950、验证集 RMSE 为 5.6267, 总运行时间仅 2.90 秒。

4.2 Adagrad

Adagrad (Adaptive Gradient) 是一种自适应梯度算法, 适用于稀疏或非平稳数据。它通过累积历史梯度平方和为每个参数分配独立学习率, 避免全局学习率过小导致的收敛慢, 或过大导致的震荡。核心思想是对梯度幅度大的参数使用较小学习率, 对小的参数使用较大学习率。

Adagrad 保留 MSE 损失 $L(w)$ 和梯度 $\nabla L(w) = \frac{2}{N} X^T (Xw - y)$, 但引入累积平方梯度矩阵 G_t :

$$G_t = G_{t-1} + (\nabla L(w_t))^2$$

更新规则为:

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \odot \nabla L(w_t)$$

其中 η 为全局学习率, $\epsilon = 10^{-8}$ 为稳定性常数, $\odot$ 为乘法。 G_t 形状与 w 相同, 累积历史平方梯度, 确保自适应性。

代码如下:

```
1 for t in range(iter_time):
2     y_pred = np.dot(x_train, w)
3     # 使用训练集计算梯度
4     gradient = 2 * np.dot(x_train.T, (y_pred - y_train)) / len(x_train)
5     # Adagrad 更新规则
6     # 1. 累积梯度的平方
7     adagrad += gradient ** 2
8     # 2. 自适应学习率更新权重
9     w -= learning_rate * gradient / np.sqrt(adagrad + eps)
```

```
Iteration 0: Train RMSE = 27.2396, Val RMSE = 26.3874
Iteration 1000: Train RMSE = 6.2169, Val RMSE = 6.0914
Iteration 2000: Train RMSE = 5.8346, Val RMSE = 5.6799
Iteration 3000: Train RMSE = 5.8037, Val RMSE = 5.6494
Iteration 4000: Train RMSE = 5.7973, Val RMSE = 5.6378
Iteration 5000: Train RMSE = 5.7956, Val RMSE = 5.6327
Iteration 6000: Train RMSE = 5.7951, Val RMSE = 5.6302
Iteration 7000: Train RMSE = 5.7950, Val RMSE = 5.6289
Iteration 8000: Train RMSE = 5.7949, Val RMSE = 5.6282
Iteration 9000: Train RMSE = 5.7949, Val RMSE = 5.6278
```

最终模型评估:

验证集 RMSE: 5.6276

总运行时间: 2.86 秒

优化算法: Adagrad

最终训练RMSE: 5.7949

最终验证RMSE: 5.6276

图 5: 使用 Adagrad 训练结果

从图 5 中可以看出, 在使用 Adagrad 时, 最终训练集 RMSE 为 5.7949、验证集 RMSE 为 5.6276, 总运行时间仅 2.86 秒。

4.3 Adam

Adam (Adaptive Moment Estimation) 结合动量法和 RMSprop 的自适应特性, 提供偏差校正机制, 提高收敛鲁棒性。它维护一阶矩 (梯度均值) 和二阶矩 (梯度平方均值) 的指数移动平均, 适用于噪声梯度环境, 如空气质量数据的时序波动。Adam 保留 MSE 损失 $L(w)$ 和梯度 $\nabla L(w)$, 引入一阶矩 m_t 和二阶矩 v_t :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla L(w_t)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla L(w_t))^2$$

偏置校正:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

更新规则:

$$w_{t+1} = w_t - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

超参数: $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ 。

代码如下:

```
1 for t in range(iter_time):
2     y_pred = np.dot(x_train, w)
```

```
3 # 使用训练集计算梯度
4 gradient = 2 * np.dot(x_train.T, (y_pred - y_train)) / len(x_train)
5 # 1. 更新一阶矩估计
6 m = beta1 * m + (1 - beta1) * gradient
7 # 2. 更新二阶矩估计
8 v = beta2 * v + (1 - beta2) * (gradient ** 2)
9 # 3. 偏差修正
10 m_hat = m / (1 - beta1 ** (t + 1))
11 v_hat = v / (1 - beta2 ** (t + 1))
12 # 4. 更新权重
13 w -= learning_rate * m_hat / (np.sqrt(v_hat) + eps)
```

```
Iteration 0: Train RMSE = 27.2396, Val RMSE = 26.3874
Iteration 1000: Train RMSE = 14.3000, Val RMSE = 16.1033
Iteration 2000: Train RMSE = 8.6725, Val RMSE = 9.2132
Iteration 3000: Train RMSE = 6.2751, Val RMSE = 6.1675
Iteration 4000: Train RMSE = 5.8395, Val RMSE = 5.6892
Iteration 5000: Train RMSE = 5.7989, Val RMSE = 5.6463
Iteration 6000: Train RMSE = 5.7951, Val RMSE = 5.6311
Iteration 7000: Train RMSE = 5.7949, Val RMSE = 5.6276
Iteration 8000: Train RMSE = 5.7949, Val RMSE = 5.6273
Iteration 9000: Train RMSE = 5.7949, Val RMSE = 5.6273
```

最终模型评估:

验证集 RMSE: 5.6275

优化算法: Adam (Momentum + RMSprop)

最终训练RMSE: 5.7949

最终验证RMSE: 5.6275

图 6: 使用 Adam 的训练结果

5 调整学习率

为优化模型的训练效率和泛化性能, 本实验针对 Adagrad 优化器系统调整学习率, 通过对比四种不同学习率的收敛过程, 选取最佳值: 使用四种 LR (0.01、0.1、0.5、1.0) 进行训练, 其他参数保持一致, 并作图讨论收敛动态。实验基于 8 个高相关性特征 (CO、NMHC、NO₂、NO_x、O₃、PM₁₀、PM_{2.5}、SO₂) 的时序窗口输入, 采用全批量 Adagrad 实现, 确保公平比较。

学习率 η 控制梯度更新步长: 在 Adagrad 的自适应框架下, η 影响有效步长 $\frac{\eta}{\sqrt{G_t + \epsilon}}$ (G_t 为累

积平方梯度)。过小 η 导致收敛缓慢，过大则引起振荡或发散。代码中，Adagrad 更新规则为：

$$w \leftarrow w - \frac{\eta \nabla L(w)}{\sqrt{G_t + \epsilon}}$$

其中 $\nabla L(w) = \frac{2}{N} X^T (Xw - y)$, $\epsilon = 10^{-8}$ 。

实验脚本独立运行四种不同学习率的模型，记录每迭代训练 MSE/RMSE 和每 1000 迭代验证 RMSE。核心循环如下：

```
1 for lr in learning_rates: # [0.01, 0.1, 0.5, 1.0]
2     w = np.zeros([dim, 1])
3     adagrad = np.zeros([dim, 1])
4     for t in range(10000):
5         y_pred = np.dot(x_train, w)
6         loss = np.mean((y_pred - y_train) ** 2)
7         gradient = 2 * np.dot(x_train.T, (y_pred - y_train)) / len(x_train)
8         adagrad += gradient ** 2
9         w -= lr * gradient / np.sqrt(adagrad + eps)
```

最终结果如下图所示：- (a) 训练 MSE： $\eta = 0.01$ （黄色）曲线最平缓，全程缓慢下降； $\eta = 0.5$ （红色）和 $\eta = 1.0$ （绿色）初期陡峭（1000 次左右迭代几乎到达最低点），后期曲线较平缓。- (b) 前 2000 迭代 Train RMSE：高 LR（红/橙）在 500 步内降 60%，突出速度；低 LR（绿/蓝）线性缓慢。- (c) 验证 RMSE：所有曲线单调下降， $\eta = 0.5$ 和 $\eta = 1.0$ 下降最快，终值最低， $\eta = 0.1$ 下降较慢， $\eta = 0.01$ 收敛最慢。- (d) 最终柱状图：学习率最低的 0.01 最高（23/21），学习率高时较矮（5.8/5.6）， $\eta = 0.5/1.0$ 并列最低。最终选取 0.5 作为最终的学习率。

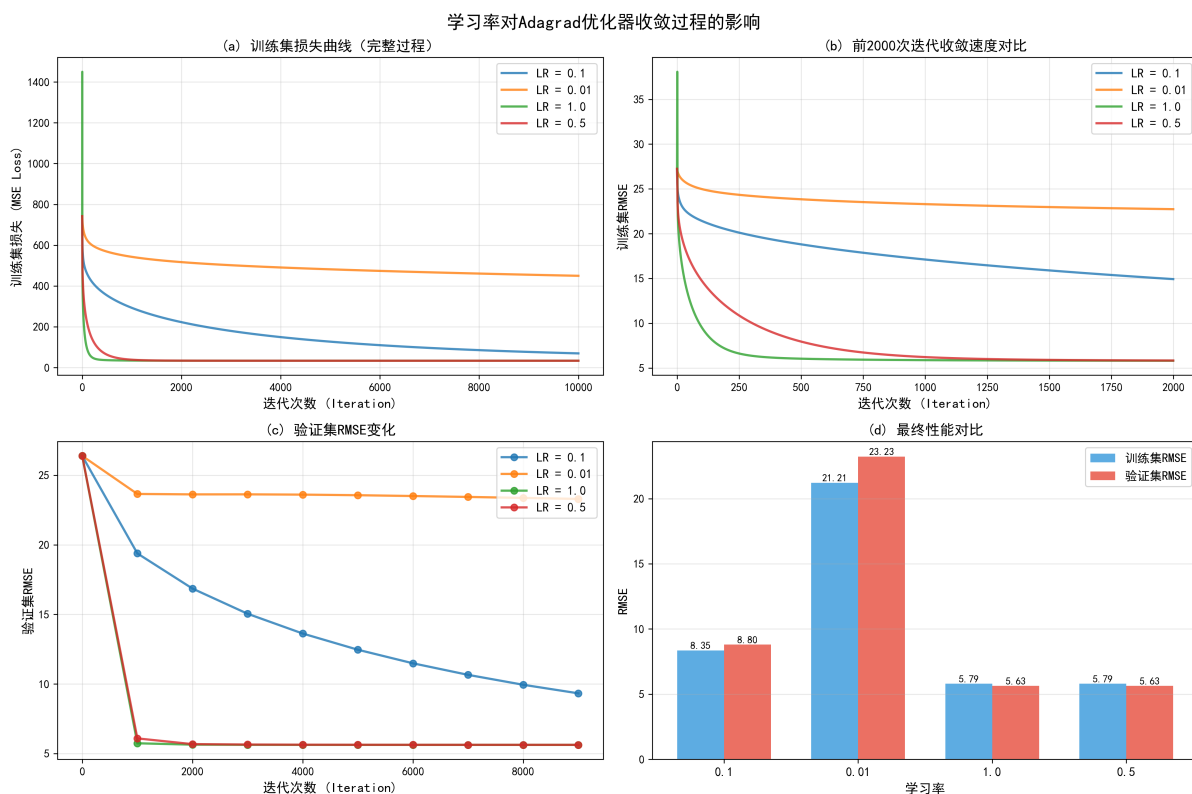


图 7：四种不同学习率的训练结果

6 正则化引入

为进一步提高模型的泛化能力和鲁棒性，本实验在 Adagrad 优化基础上引入 L2 正则化，针对高维特征防止过拟合，进一步提升准确度。L2 正则化在损失函数中添加权重惩罚项，收缩参数估计，减少噪声敏感性，尤其适用于空气质量数据的季节波动和缺失值。

L2 正则化的总损失函数为：

$$L(w) = \frac{1}{N} \|y - Xw\|_2^2 + \lambda \|w\|_2^2 = \text{MSE} + \lambda \sum_{j=1}^d w_j^2$$

其中 $\lambda = 0.01$ 为正则化系数， w_0 （偏置）排除惩罚（ $\|w[1:] \|_2^2$ ）。梯度相应调整：

$$\nabla L(w) = \frac{2}{N} X^T (Xw - y) + 2\lambda w \quad (\text{偏置项 } \nabla_{\text{reg}} = 0)$$

Adagrad 更新融入总梯度：

$$w \leftarrow w - \frac{\eta \nabla L(w)}{\sqrt{G_t} + \epsilon}$$

具体代码如下：

```
1 for t in range(iter_time):
2     # 在训练集上计算预测和损失
3     y_pred = np.dot(x_train, w)
4     # L2正则化项
5     reg_term = lambda_reg * np.sum(w[1:] ** 2) # 跳过偏置项w[0]
6     total_loss = mse_loss + reg_term
7     reg_losses.append(total_loss)
8
9     # MSE的梯度
10    gradient_mse = 2 * np.dot(x_train.T, (y_pred - y_train)) / len(x_train)
11    # L2正则化的梯度
12    gradient_reg = np.zeros_like(w)
13    gradient_reg[1:] = 2 * lambda_reg * w[1:]
14    # 总梯度 = MSE梯度 + 正则化梯度
15    gradient_total = gradient_mse + gradient_reg
16    adagrad += gradient_total ** 2
17    w -= learning_rate * gradient_total / np.sqrt(adagrad + eps)
```

结果如下图所示，从图中可以看出，在使用 Adagrad 梯度下降和 L2 正则化时，最终训练集 RMSE 为 5.8453、验证集 RMSE 为 5.7378，总运行时间仅 3.07 秒。

```
Iter      0: Train RMSE = 27.2396, Val RMSE = 26.3874, Reg = 0.0000
Iter  1000: Train RMSE =  6.2578, Val RMSE =  6.1544, Reg = 1.4361
Iter  2000: Train RMSE =  5.8880, Val RMSE =  5.7784, Reg = 1.6305
Iter  3000: Train RMSE =  5.8571, Val RMSE =  5.7548, Reg = 1.6955
Iter  4000: Train RMSE =  5.8493, Val RMSE =  5.7450, Reg = 1.7182
Iter  5000: Train RMSE =  5.8468, Val RMSE =  5.7407, Reg = 1.7264
Iter  6000: Train RMSE =  5.8459, Val RMSE =  5.7390, Reg = 1.7294
Iter  7000: Train RMSE =  5.8455, Val RMSE =  5.7383, Reg = 1.7305
Iter  8000: Train RMSE =  5.8454, Val RMSE =  5.7380, Reg = 1.7309
Iter  9000: Train RMSE =  5.8453, Val RMSE =  5.7379, Reg = 1.7311
```

最终模型评估：

总运行时间：3.07 秒

优化算法：Adagrad (自适应学习率)

正则化：L2正则化 ($\lambda=0.01$)

学习率：0.5

最终训练RMSE：5.8453

最终验证RMSE：5.7378

权重L2范数：17.3119

图 8: 加入 L2 正则化后的训练结果

7 结果分析与评估

为了更好地对实验进行评估，将几个实验的 RMSE 曲线可视化并放入同一个图中，分别为：使用 baseline 模型、使用全部 18 个特征和 BGD 梯度下降技术、使用相关性最高的 8 个特征和 BGD 梯度下降技术、使用相关性最高的 8 个特征和 Adagrand 梯度下降技术、使用相关性最高的 8 个特征和 Adam 梯度下降技术、使用相关性最高的 8 个特征和 Adagrand 梯度下降技术及 L2 正则化。结果如下两图所示，画出在训练集和验证集上的 RMSE 收敛曲线。

综合收敛表现：Adagrad 在早期下降速度显著快于 BGD，而 Adam 收敛较慢，最终结果均相近；高相关 8 特征显著好于仅使用 PM2.5，且比“全特征”运行速度更快。

验证集趋势：Adagrad、Adam 验证 RMSE 更低，表明更好的泛化；带 L2 的 Adagrad 进一步缩小 train-val 差距。

最终性能对比：“高相关 + Adagrad”在验证 RMSE 与 gap 上表现均衡，推荐为本任务的首选组合。

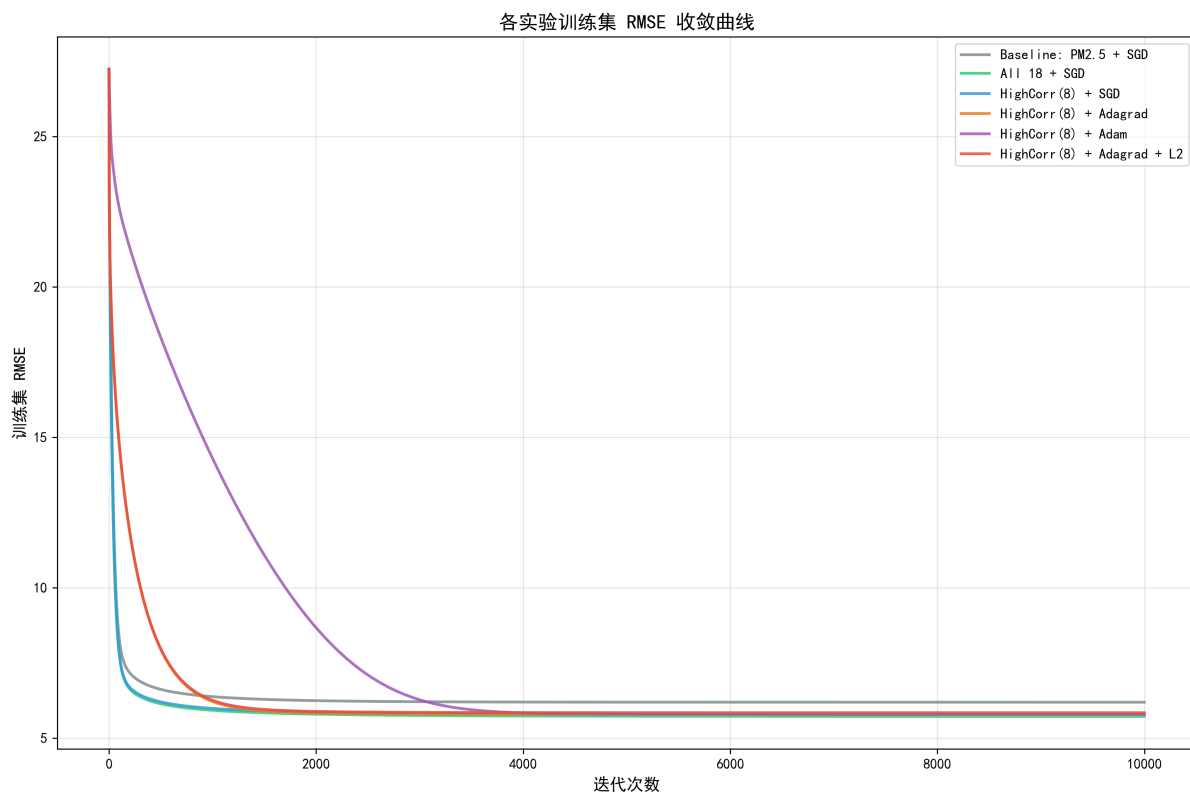


图 9: 各实验训练集 RMSE 变化

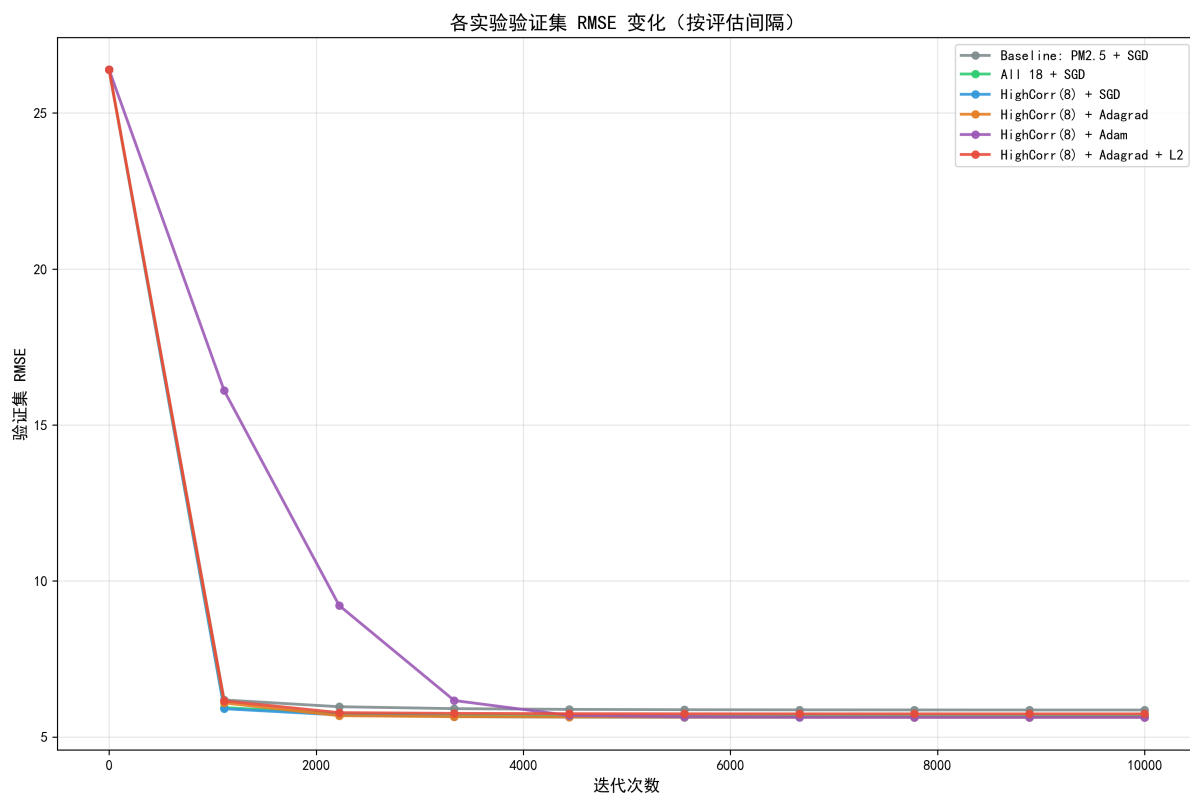


图 10: 各实验验证集 RMSE 变化

8 结论

本研究以“由简入深”的路线优化 PM2.5 预测的线性回归模型：从仅用 PM2.5 的最简单模型出发，依次扩展到全特征、使用 Pearson 相关系数筛选高相关 8 特征，再比较 BGD、Adagrad、Adam 等优化器，并进行学习率对比与 L2 正则化实验。全流程统一采用滑动窗构样本、逐维标准化、80/20 训练集和测试集划分与 10,000 次迭代，以验证集 RMSE 作为主指标，并通过可视化对训练/验证收敛过程与最终性能进行评估。

实验结果显示，高相关 8 特征在信息密度与泛化之间取得更好平衡；Adagrad 相较 SGD 收敛更快且更稳定；在 Adagrad 下学习率 0.5–1.0 综合表现最佳；引入 L2 正则化显著降低权重范数并缩小 train-val gap，有助于抑制过拟合。综合速度、稳定性与验证性能，最终使用的模型为：使用最高相关八个特征、Adagrad 梯度下降技术、学习率为 0.5、使用 L2 正则化。

9 附录

- exp1_only2.5.py: 仅使用 PM2.5 作为特征、BGD 梯度下降技术
- exp2_all_features.py: 选择所有作为特征、BGD 梯度下降技术
- exp3_high_corr_features.py: 选择相关性最高的八个特征、BGD 梯度下降技术
- exp4_high_corr_adagrad.py: 选择相关性最高的八个特征、Adagrd 梯度下降技术
- exp5_high_corr_adam.py: 选择相关性最高的八个特征、Adam 梯度下降技术
- exp6_high_corr_adagrad_regularization.py: 选择相关性最高的八个特征、Adagrd 梯度下降技术、L2 正则化
- exp_learning_rate_comparison.py: 不同学习率比较、Adagrd 梯度下降技术
- result.csv: 结果文件